

Universidade de Brasília

Orientação por Objetos



TRABALHO PRÁTICO
Sistema de Mobilidade Urbana (Ride-Sharing)

André Luiz Peron Martins Lanna - Professor

João Aquila Teixeira de Alencar - 242023980

Luís Henrique Luna de Arruda - 242004869

Marco Antonio Araujo Roman - 241025309

Rafael Souto Lopes Laube - 211062428

Brasília
8 de dezembro de 2025

8 de dezembro de 2025

Conteúdo

1	Visão Geral	3
2	Entidades	3
2.1	Usuário	3
2.2	Subclasses principais: Passageiro e Motorista	4
2.2.1	Passageiro	4
2.2.2	Motorista	4
2.3	Outras entidades	4
3	Enums	5
4	Associações e UML	6
5	Herança	6
6	Polimorfismo	6
7	Modularidade	6
7.1	Repositórios	6
7.1.1	Paramétricos	6
7.2	Controllers	7
7.2.1	MotoristaController	7
7.2.2	PassageiroController	7
7.3	Serviços	8
8	Regras de Negócio	8
8.1	Fluxo de corrida	8
8.2	Estado de corrida	9
8.3	Disponibilidade de motorista	9
8.4	Validações de negócio	10

9 Tratamento de exceções	10
10 Implementação em Java	11
11 Diagrama de classes UML	11
12 Conclusão	15

1 Visão Geral

Neste documento apresentamos o desenvolvimento do nosso modelo de aplicativo de corridas. O objetivo é descrever, de maneira clara e estruturada, todo o conjunto de processos envolvidos no funcionamento do sistema: autenticação do usuário, solicitação de corrida, aceite por parte do motorista, execução da viagem, finalização e pagamento.

A arquitetura do projeto foi dividida em camadas com responsabilidades bem definidas. Entre elas, destacam-se os **Enums**, que representam os diferentes estados da corrida e do motorista; os **Controllers**, responsáveis por interpretar as ações do usuário e aplicar as validações necessárias; e a interface de Front-end, que recebe os comandos e gerencia listas de usuários, garantindo interação visual e navegação adequadas.

A implementação faz uso dos conceitos estudados em Programação Orientada a Objetos, como classes, abstração, encapsulamento e hierarquias que permitem reutilização de código e organização lógica. Também aplicamos polimorfismo paramétrico por meio de Generics, principalmente para estruturar listas de usuários no processo de login. Outro ponto fundamental foi o uso de **Exceptions**, aprendidas em aula, utilizadas para impedir operações inválidas, sinalizando erros e garantindo consistência no fluxo do sistema.

O resultado é um aplicativo funcional que integra todos os conteúdos estudados durante o semestre, representando não apenas os aspectos técnicos da disciplina, mas também a lógica e o comportamento esperados de um sistema real de mobilidade urbana.

2 Entidades

2.1 Usuário

No início do projeto, definimos a classe **Usuario** como uma classe abstrata. Ela serve como base para as duas subclasses principais, compartilhando atributos e comportamentos comuns, como dados de autenticação (e-mail, senha e nome) e o sistema de avaliação, utilizado tanto pelo passageiro quanto pelo motorista. Além disso, utilizamos getters e setters para permitir a modificação controlada desses atributos durante o uso do sistema, respeitando o encapsulamento.

2.2 Subclasses principais: Passageiro e Motorista

2.2.1 Passageiro

A classe **Passageiro** é responsável por gerenciar o cadastro do usuário e seu meio de pagamento por meio do método `cadastrarMeioDePagamento()`. Além disso, o passageiro é capaz de solicitar corridas, funcionalidade que será detalhada posteriormente no documento.

O passageiro também possui responsabilidades importantes, como adicionar um meio de pagamento antes de solicitar uma corrida.

Alguns desses métodos estão implementados em outras entidades do sistema, que serão explicadas em seções posteriores.

2.2.2 Motorista

A classe **Motorista** possui responsabilidades específicas, entre elas:

- indicar quando está disponível para receber solicitações de corrida;
- fornecer dados necessários para validação, como CNH e informações do veículo.

O motorista também coordena outras entidades do sistema, funcionando de forma semelhante ao passageiro, mas lidando com um conjunto maior de validações e informações.

2.3 Outras entidades

A classe **Veiculo** representa o automóvel que o motorista vai dirigir, guardando as informações para diferenciar ele dos outros e de tornar mais fácil a identificação para o passageiro na hora de embarcar, aqui vemos algumas de suas características:

- placa;
- modelo;
- cor;
- ano;

Cada motorista pode possuir diversos veículos cadastrados, mas deve ter um veículo ativo por vez, garantindo que não haja confusões na hora de começar uma corrida ou dificultar a identificação por parte do passageiro.

O sistema utiliza uma entidade genérica chamada de **MetodoPagamento** projetada para ser um hub e utilizando de polimorfismo, abrigando toda a lógica dos métodos de pagamento que possibilitamos cadastrar para serem usados. Cada forma de pagamento tem seu código próprio e sua própria lógica, mas a comunicação entre o meio de pagamento é feita por essa entidade **MetodoPagamento**

A entidade central do sistema é a **Corrida**, conectando o passageiro com o motorista e mostrando informações do veículo, além de controlar o ciclo de vida da viagem, incluem atributos como

1. ponto de partida e destino
2. categoria da corrida
3. status da corrida
4. preço estimado
5. status da corrida

3 Enums

O pacote de Enumeradores foi criado para padronizar as opções fixas do sistema. O uso de Enums evita erros, como digitar "luxo" com letra minúscula ou números inválidos, e torna o código mais organizado. Os Enums que usamos são:

- **CategoriaCorridaEnum**: Define os tipos de serviço oferecidos pelo aplicativo, limitando as opções a Comum ou Luxo;
- **CategoriaUsuarioEnum**: Serve para identificar o papel de quem está usando o sistema, separando Motoristas de Passageiros;
- **NotaEnum**: Garante que as avaliações sejam feitas apenas com valores válidos, de 1 a 5. Este enum possui uma característica especial, ele guarda o valor numérico correspondente à nota, facilitando cálculos futuros;
- **StatusCorridaEnum**: Controla as etapas da viagem; ele indica se a corrida foi Solicitada, Aceita, está Em Andamento ou se foi Concluída/Cancelada;
- **StatusMotoristaEnum**: Indica a disponibilidade do motorista em tempo real, informando se ele está Online, Offline ou Em Corrida.

4 Associações e UML

Na UML e na Programação Orientada a Objetos, uma associação define um relacionamento estrutural entre classes, ela descreve como objetos de uma classe se conectam e interagem com objetos de outra classe.

Em termos simples: se você tem uma classe Motorista e uma classe Carro, e o Motorista possui um carro, existe uma associação entre eles.

5 Herança

A Herança, que também é chamada de generalização, é o mecanismo que permite que uma subclasse herde características, atributos, e comportamentos, métodos, de outra classe já existente, chamada de Superclasse ou "Classe-mãe"

6 Polimorfismo

Polimorfismo = propriedade, estado ou comportamento que se apresenta de várias formas diferentes. Polimorfismo é uma propriedade fundamental em Orientação por Objetos, pois permite que comportamentos ou propriedades sejam definidos para cada classe/objeto.

7 Modularidade

7.1 Repositórios

7.1.1 Paramétricos

A classe `Cadastro<T>` é uma estrutura genérica responsável por guardar e organizar uma lista de itens de qualquer tipo (`T`). Ela permite usar a mesma lógica para qualquer tipo de objeto, evitando ter que escrever o mesmo código várias vezes. Esta classe desempenha um papel similar ao de um `ArrayList` e aplica algumas restrições simples, como o limite máximo de registros definido por `tamanhoMax`. As principais responsabilidades do cadastro incluem:

- `adicionar(T t)`: Insere um novo elemento na lista, verificando previamente se a capacidade máxima de armazenamento foi atingida;
- `buscarPorId(int id)`: Recupera um objeto específico com base em seu índice na lista, tratando exceções de índices inválidos;

- `remove(Object x)`: Remove um objeto existente da lista.

Dessa forma, a classe `Cadastro` serve como base para a criação de repositórios específicos (como `Cadastro de Passageiros` ou `Cadastro de Motoristas`) sem a necessidade de reescrever a lógica de manipulação da lista.

Além de permitir a reutilização da lógica de cadastro, o uso de `Generics` fortalece o polimorfismo paramétrico em Java, garantindo que cada repositório manipule apenas os tipos corretos. Assim, o sistema evita erros de tipo em tempo de execução e ganha mais segurança e organização interna.

7.2 Controllers

7.2.1 MotoristaController

O `MotoristaController` implementa essa regra de negócio diretamente no método `ficarOnline`. Antes de permitir que o status do motorista mude para `ONLINE`, o código realiza uma verificação crucial:

Verificação de Veículo Ativo: O código acessa `motorista.getVeiculoAtivo()`. Isso confirma que o sistema respeita a regra de que o motorista precisa ter selecionado um veículo específico dentre os seus cadastrados para começar a trabalhar.

Validação de Documentação: Além de verificar se o veículo existe, o controlador verifica `isDocumentacaoValida()`. Isso garante que as informações para diferenciar ele dos outros, como placa, modelo, entre outros, estão aptas para rodar.

Se o motorista não tiver um veículo ativo e válido, o controller lança a `MotoristaInvalidoException`, impedindo a operação.

7.2.2 PassageiroController

Este controlador gerencia duas entidades, a `Corrida` e o `MetodoPagamento`.

A entidade `Corrida` serve para controlar o ciclo de vida da viagem. No método `processarPagamento` do `PassageiroController`, vemos essa regra sendo aplicada através da verificação de status (`StatusCorridaEnum status = corrida.getStatus()`). Isso impede que o ciclo de vida seja quebrado, um passageiro não pode iniciar ou pagar uma nova interação se o estado atual da corrida ainda estiver ativo (`EM_ANDAMENTO` ou `ACEITA`).

O `MetodoPagamento` serve como um "hub" abrigando toda a lógica dos métodos de pagamento. O controlador mostra isso ao tratar o pagamento de forma genérica e polimórfica:

- Validação de Existência: O código verifica `if (meioPadrao == null)` e lança `MetodoPagamentoInexistenteException`, isso garante que o "hub" de pagamento foi instanciado corretamente.

- Verificação de Saldo: Antes de acionar a lógica específica do método de pagamento, que estaria dentro das subclasses de `MeioDePagamento` via polimorfismo, o controller atua como uma barreira de segurança, verificando se o saldo é menor que `valorParaPagar`.

7.3 Serviços

A camada de Serviços atua como uma ponte entre os Controladores e os Repositórios. Enquanto os Controladores gerenciam a interação direta com o usuário, recebendo comandos, os Serviços encapsulam a lógica "pesada" do negócio.

No nosso sistema, classes como `CorridaService` são responsáveis por orquestrar ações complexas que envolvem múltiplas entidades. Por exemplo, ao solicitar uma corrida, o serviço não apenas cria o objeto `Corrida`, mas também busca motoristas disponíveis na lista de cadastros e calcula a estimativa de preço baseada na distância e na categoria, comum ou luxo. Essa separação garante que o código permaneça modular e que as regras de negócio não fiquem espalhadas na camada de interface.

8 Regras de Negócio

As regras de negócio do sistema definem o funcionamento lógico do aplicativo de mobilidade urbana. Elas determinam em quais condições uma corrida pode ser solicitada, aceita, iniciada, concluída ou paga, além das validações exigidas para que motoristas e passageiros possam executar suas ações. Todas essas regras foram extraídas diretamente da implementação dos controladores do sistema.

8.1 Fluxo de corrida

O fluxo principal do sistema segue uma ordem lógica de estados para garantir a consistência da operação:

1. **Solicitação:** O Passageiro informa origem, destino e categoria. O sistema valida se há saldo/meio de pagamento e cria a corrida com status `SOLICITADA`.

2. **Aceite:** Um Motorista disponível, status ONLINE, visualiza a solicitação. Ao aceitar, o status da corrida muda para ACEITA e o do motorista para EM_CORRIDA.
3. **Em Andamento:** Quando o motorista busca o passageiro e inicia o trajeto, a corrida passa para EM_ANDAMENTO.
4. **Finalização:** Ao chegar ao destino, o motorista encerra a viagem. O status muda para CONCLUIDA.
5. **Pagamento e Avaliação:** O sistema processa o pagamento automaticamente e libera a opção para que ambas as partes se avaliem, nota 1 a 5.

8.2 Estado de corrida

O ciclo de vida da viagem é rigidamente controlado pelo StatusCorridaEnum. As transições permitidas são unidirecionais para evitar inconsistências, exemplo: uma corrida concluída não pode voltar a ser solicitada.

- **SOLICITADA:** Aguardando motorista.
- **ACEITA:** Motorista a caminho do ponto de embarque.
- **EM_ANDAMENTO:** Trajeto sendo realizado.
- **CONCLUIDA:** Viagem finalizada com sucesso.
- **CANCELADA:** Viagem interrompida pelo usuário ou motorista antes da conclusão.

8.3 Disponibilidade de motorista

A gestão de disponibilidade é feita através do StatusMotoristaEnum. Para um motorista receber corridas, ele deve alterar explicitamente seu status para ONLINE. O sistema impede que um motorista receba novas chamadas se estiver EM_CORRIDA ou OFFLINE. Além disso, a mudança automática de status ocorre quando uma corrida é aceita, bloqueando o motorista para novas solicitações até que a atual seja finalizada.

8.4 Validações de negócio

Para garantir a integridade dos dados e a segurança da operação, implementamos diversas validações:

- **Saldo Insuficiente:** Antes de iniciar uma corrida, o sistema verifica se o passageiro possui saldo ou limite no cartão.
- **Veículo Ativo:** Um motorista não pode ficar ONLINE sem ter selecionado um veículo ativo na sua lista de bens.
- **Documentação:** A CNH do motorista e o documento do veículo devem estar marcados como válidos no sistema.
- **Notas Válidas:** O sistema de avaliação rejeita qualquer input numérico que não esteja contido no conjunto $\{1, 2, 3, 4, 5\}$.

9 Tratamento de exceções

Utilizamos exceções personalizadas (Custom Exceptions) para gerenciar erros de lógica de negócio de forma elegante, evitando que o programa feche abruptamente e permitindo mensagens de erro claras ao usuário. As principais exceções implementadas foram:

- `MotoristaInvalidoException`: Lançada quando um motorista tenta ficar online sem cumprir os requisitos (documentação vencida ou sem veículo).
- `MetodoPagamentoInexistenteException`: Disparada se o passageiro tentar solicitar uma corrida sem ter cadastrado um cartão ou PIX.
- `SaldoInsuficienteException`: Ocorre na etapa de validação financeira.
- `OpcaoInvalidaException`: Utilizada nos menus do sistema para tratar entradas incorretas do usuário.

Todas as exceções são capturadas em blocos try-catch nas classes Controller ou na Main, garantindo que o fluxo do programa retorne ao menu principal em vez de travar.

10 Implementação em Java

O projeto foi desenvolvido utilizando Java, focando puramente nos pilares da Orientação a Objetos, com a implementação de uma interface gráfica programada em javascript, conforme requisito da disciplina. A estrutura do projeto foi organizada em pacotes para melhor modularização, dentre eles os principais são:

- **entidades:** Contém as entidades (Usuario, Motorista, Passageiro, Veiculo, Corrida).
- **enums:** Contém as constantes do sistema (Status, Categoria).
- **parametricos:** Contém a classe genérica `Cadastro<T>` e suas especializações.
- **controller:** Responsável por receber o input do usuário e chamar os métodos das entidades.
- **services:** Contém os serviços do sistema.
- **exceptions:** Contém as classes de erro personalizadas

O uso de Generics na classe `Cadastro<T>` foi um dos pontos altos da implementação, permitindo que a mesma lógica de adicionar, remover e buscar fosse reutilizada tanto para Motoristas quanto para Passageiros, reduzindo drasticamente a duplicação de código.

11 Diagrama de classes UML

O diagrama de classes do sistema, mostrado na imagem abaixo, reflete as relações de herança e associação discutidas:

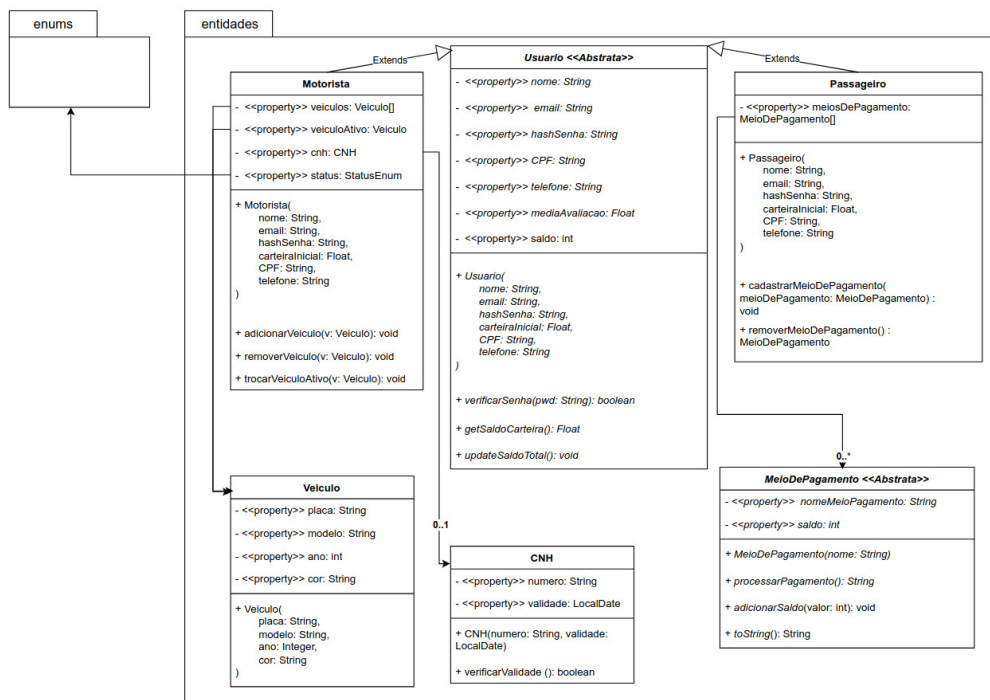


Figura 1: Representação estrutural das classes do sistema.

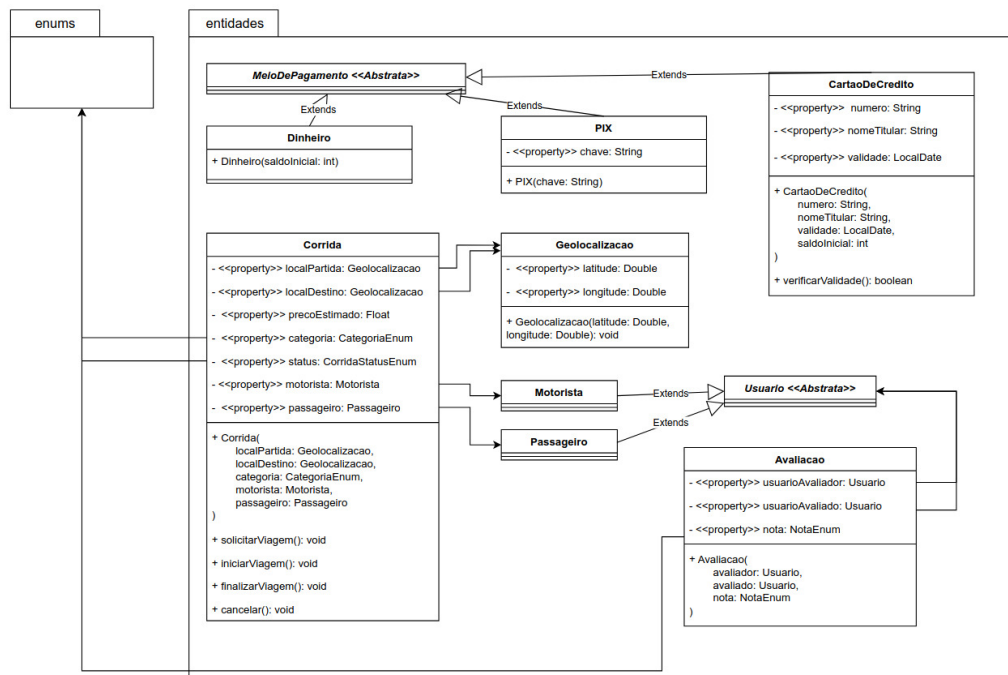


Figura 2: Representação estrutural das classes do sistema.

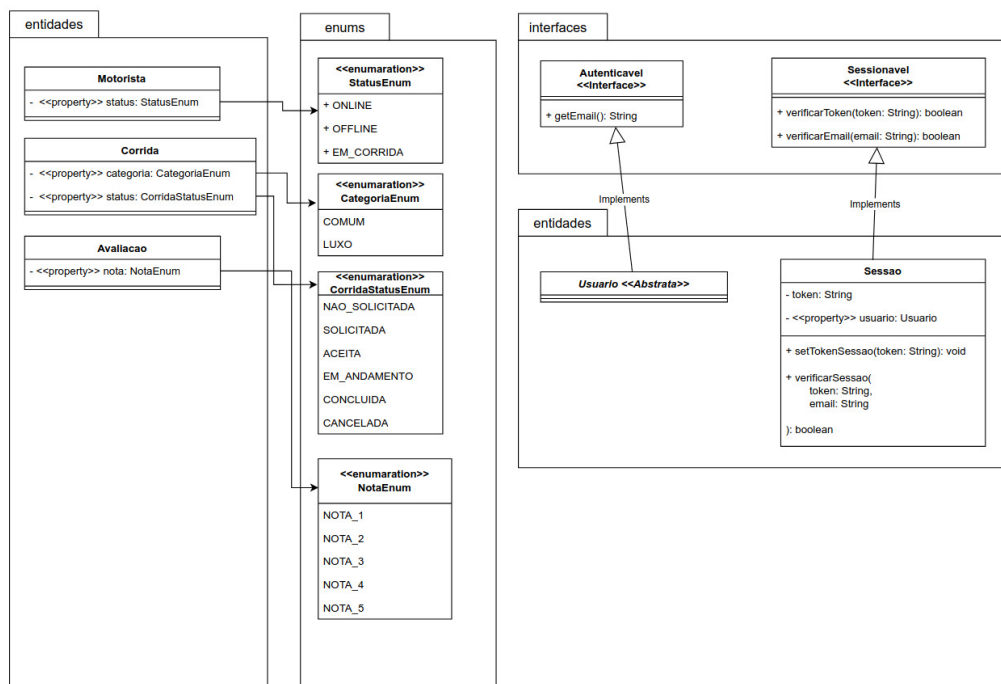


Figura 3: Representação estrutural das classes do sistema.

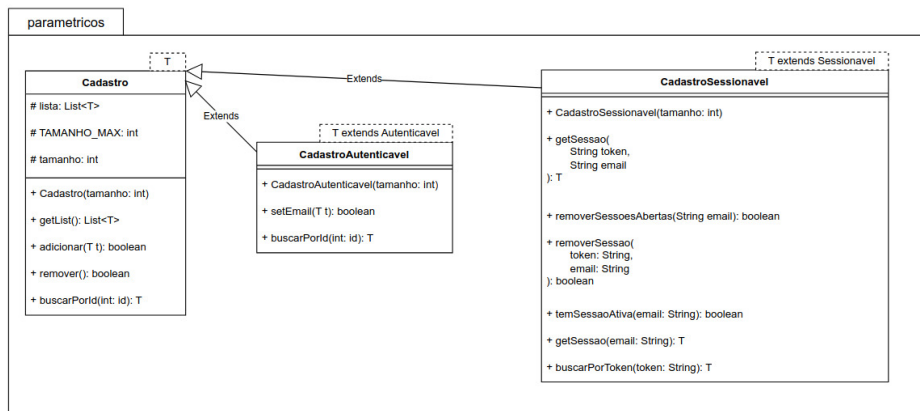


Figura 4: Representação estrutural das classes do sistema.

12 Conclusão

O desenvolvimento do Sistema de Mobilidade Urbana permitiu a aplicação prática dos conceitos fundamentais da Orientação a Objetos. Através da herança, conseguimos reaproveitar códigos comuns a todos os usuários na classe mãe. O encapsulamento garantiu a segurança dos dados sensíveis, como senhas e saldos, acessíveis apenas via métodos autorizados. O polimorfismo foi essencial tanto no tratamento genérico dos métodos de pagamento quanto na implementação flexível dos repositórios com Generics.

Além dos aspectos técnicos, o trabalho evidenciou a importância de uma boa modelagem inicial, UML, e do tratamento robusto de exceções para criar um software resistente a falhas. O sistema desenvolvido simula as principais funções de um aplicativo de transporte real.